

REST API Exercise

A REST API (Representational State Transfer Application Programming Interface) allows structured access to data over HTTP. Unlike web scraping, APIs provide a defined, stable interface with structured responses (typically JSON), making data retrieval more reliable and efficient. Many public institutions and organizations offer open APIs to share data without requiring authentication.

This notebook retrieves economic indicators for Germany from the **World Bank Open Data API** — a free, publicly accessible REST API that requires no API key.

Use Python, `requests`, and `pandas` to retrieve data from a REST API:

Import Libraries

```
In [5]: import requests
import pandas as pd
import matplotlib.pyplot as plt
```

Define the Target API

The World Bank Open Data API provides free access to economic indicators for countries worldwide. No API key is required.

- **Base URL:** `https://api.worldbank.org/v2/`
- **Country code:** `DE` (Germany)
- **Indicators used:**
 - `NY.GDP.PCAP.CD` – GDP per capita (current USD)
 - `SL.UEM.TOTL.ZS` – Unemployment rate (% of total labor force)
 - `SP.POP.TOTL` – Total population
- **Documentation:**
<https://datahelpdesk.worldbank.org/knowledgebase/articles/889392>

```
In [6]: BASE_URL = "https://api.worldbank.org/v2/country/DE/indicator/{indicator}"

INDICATORS = {
    "NY.GDP.PCAP.CD": "GDP_per_capita_USD",
    "SL.UEM.TOTL.ZS": "Unemployment_rate_pct",
    "SP.POP.TOTL":    "Population"
}
```

Send Requests to the API

Each indicator is retrieved with a separate GET request. The API returns paginated JSON responses — `per_page=100` ensures all years are returned in a single request. The

response status code is checked before processing.

```
In [9]: def fetch_indicator(indicator_code, label):
        url = BASE_URL.format(indicator=indicator_code)
        params = {
            "format": "json",
            "per_page": 100,
            "date": "1993:2025"
        }

        response = requests.get(url, params=params)
        print(f"[{label}] Status Code: {response.status_code}", end=" - ")

        if response.status_code == 200:
            print("OK")
            data = response.json()
            return data[1] # data[0] = metadata, data[1] = records
        else:
            print("FAILED")
            return []

        raw_results = {}
        for code, label in INDICATORS.items():
            raw_results[label] = fetch_indicator(code, label)
```

[GDP_per_capita_USD] Status Code: 200 – OK
 [Unemployment_rate_pct] Status Code: 200 – OK
 [Population] Status Code: 200 – OK

Parse the JSON Response

The API returns a list of records, each containing the year (`date`) and the value. Records with missing values (`None`) are kept at this stage and handled in the structuring step.

```
In [10]: def parse_records(records, value_column):
        parsed = []
        for record in records:
            year = int(record["date"])
            value = record["value"]
            parsed.append({"Year": year, value_column: value})
        return parsed

        # Show a sample of raw JSON for one indicator
        print("Sample raw JSON record:")
        print(raw_results["GDP_per_capita_USD"][0])
```

Sample raw JSON record:

```
{'indicator': {'id': 'NY.GDP.PCAP.CD', 'value': 'GDP per capita (current US$)'},
 'country': {'id': 'DE', 'value': 'Germany'}, 'countryiso3code': 'DEU', 'date': '2025', 'value': 60496.4350820414, 'unit': '', 'obs_status': '', 'decimal': 1}
```

Identify the Data to be Retrieved

I want to retrieve three economic indicators for Germany covering the years 1993 to 2025 from the World Bank Open Data API: GDP per capita (in USD), the unemployment rate (in percent), and the total population. These indicators will be combined into a single

structured dataset to complement the beer consumption and inflation data from the other exercises, enabling a broader economic analysis.

Extract and Combine Data

Each indicator is parsed into a separate DataFrame and then merged on the **Year** column into a single dataset covering 1993–2025.

```
In [11]: dfs = []
for label, records in raw_results.items():
    parsed = parse_records(records, label)
    df_temp = pd.DataFrame(parsed)
    dfs.append(df_temp)

# Merge all indicators on Year
df_api = dfs[0]
for df_temp in dfs[1:]:
    df_api = df_api.merge(df_temp, on="Year", how="outer")

df_api = df_api.sort_values(by="Year").reset_index(drop=True)

print(f"Data successfully extracted. Found rows: {len(df_api)}")
df_api.head(10)
```

Data successfully extracted. Found rows: 33

```
Out[11]:
```

	Year	GDP_per_capita_USD	Unemployment_rate_pct	Population
0	1993	25616.650877	7.675	81156363
1	1994	27201.959356	8.727	81438348
2	1995	31747.244940	8.158	81678051
3	1996	30599.789105	8.822	81914831
4	1997	27046.956546	9.861	82034771
5	1998	27395.944061	9.787	82047195
6	1999	26965.492277	8.854	82100243
7	2000	23925.855990	7.917	82211508
8	2001	23878.364147	7.773	82349925
9	2002	25486.594201	8.482	82488495

Store Data in a Structured Format

Give a brief overview of the data collected (e.g. count, fields, ...).

```
In [12]: print("--- Overview of API data ---")
print(f"Number of rows (years): {df_api.shape[0]}")
print(f"Column names: {list(df_api.columns)}")
print(f"Year range: {df_api['Year'].min()} - {df_api['Year'].max()}")
print(f"Missing values per column:")
```

```
print(df_api.isnull().sum())
print()
print("Data Structure Overview:")
df_api.info()
```

--- Overview of API data ---

Number of rows (years): 33

Column names: ['Year', 'GDP_per_capita_USD', 'Unemployment_rate_pct', 'Population']

Year range: 1993 - 2025

Missing values per column:

Year 0

GDP_per_capita_USD 0

Unemployment_rate_pct 0

Population 0

dtype: int64

Data Structure Overview:

<class 'pandas.DataFrame'>

RangeIndex: 33 entries, 0 to 32

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	Year	33 non-null	int64
1	GDP_per_capita_USD	33 non-null	float64
2	Unemployment_rate_pct	33 non-null	float64
3	Population	33 non-null	int64

dtypes: float64(2), int64(2)

memory usage: 1.2 KB

Visualize Data

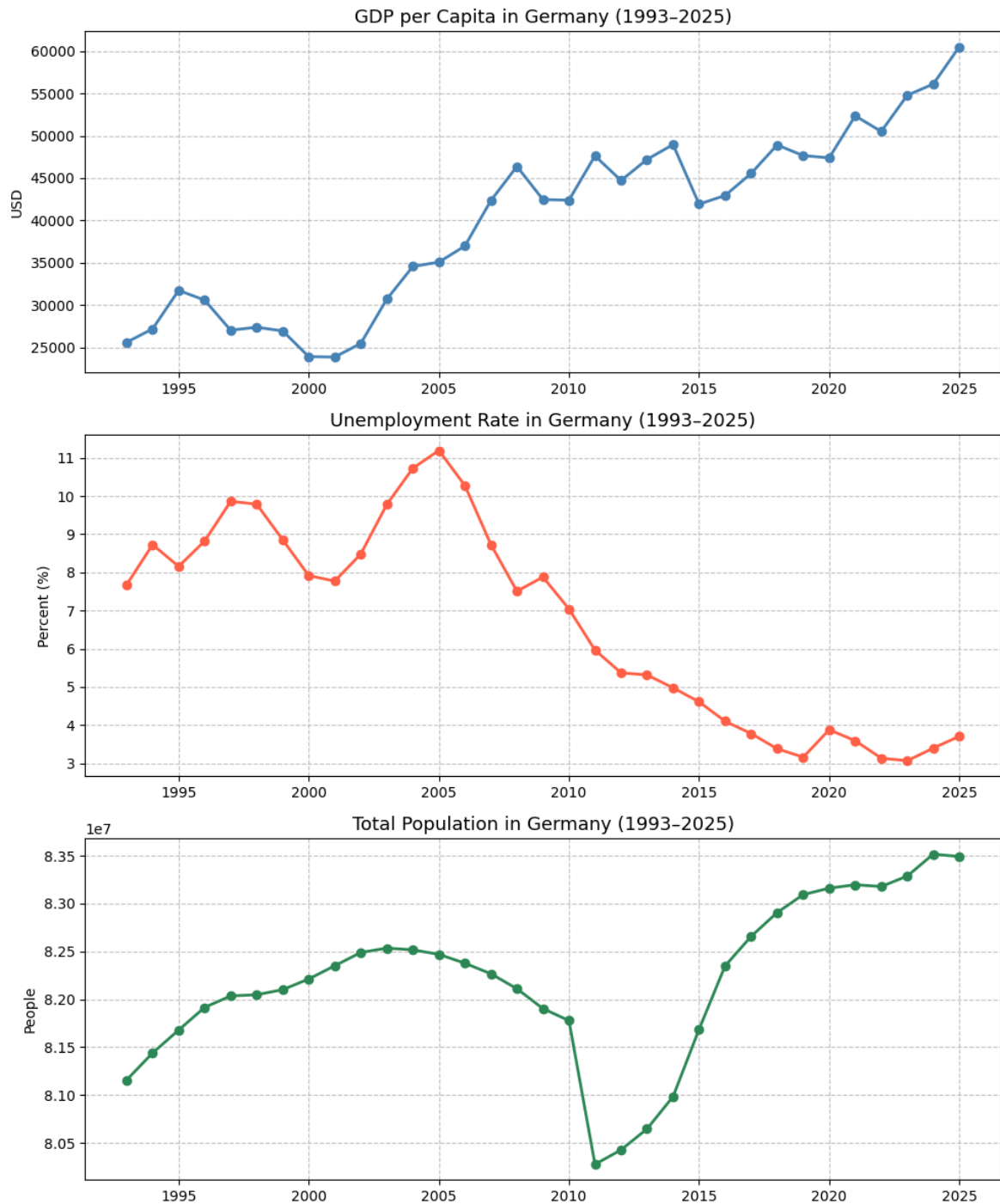
```
In [13]: fig, axes = plt.subplots(3, 1, figsize=(10, 12))

axes[0].plot(df_api['Year'], df_api['GDP_per_capita_USD'], marker='o', color='steelblue')
axes[0].set_title('GDP per Capita in Germany (1993-2025)', fontsize=13)
axes[0].set_ylabel('USD')
axes[0].grid(True, linestyle='--', alpha=0.7)

axes[1].plot(df_api['Year'], df_api['Unemployment_rate_pct'], marker='o', color='coral')
axes[1].set_title('Unemployment Rate in Germany (1993-2025)', fontsize=13)
axes[1].set_ylabel('Percent (%)')
axes[1].grid(True, linestyle='--', alpha=0.7)

axes[2].plot(df_api['Year'], df_api['Population'], marker='o', color='seagreen',
             linestyle='--', alpha=0.7)
axes[2].set_title('Total Population in Germany (1993-2025)', fontsize=13)
axes[2].set_ylabel('People')

plt.tight_layout()
plt.show()
```



Save the Data

```
In [14]: df_api.to_csv('world_bank_data.csv', index=False)
print("Data successfully saved as 'world_bank_data.csv'")
```

Data successfully saved as 'world_bank_data.csv'